



Ad Soyad: _____

Numara: _____

Tarih: _____

1 Çalışma 1: Coyote Time + Jump Buffer

Örnek Kod

Aşağıdaki iskeleti `calisma1.py` dosyasına yazın. Amacı coyote time ve jump buffer mekaniklerini işleten bir platformer:

```
1 import pygame
2
3 TILE = 32
4 GENISLIK, YUKSEKLIK = 640, 480
5 COYOTE_SURE = 6
6 BUFFER_SURE = 6
7 ZIP_HIZI = -11
8 YERCEKIMI = 0.5
9
10 HARITA = [
11     "                ",
12     "                ",
13     "          22      ",
14     "                ",
15     "    222          22 ",
16     "                ",
17     "          222      ",
18     "2                ",
19     "22222222 222222222 ",
20     "222222222222222222 ",
21 ]
22
23 def katilari_cikar(harita):
24     katilar = []
25     for r, s in enumerate(harita):
26         for c, ch in enumerate(s):
27             if ch == "2":
28                 katilar.append(pygame.Rect(c*TILE, r*TILE, TILE, TILE))
29     return katilar
30
31 class Oyuncu:
32     def __init__(self, x, y):
33         self.rect = pygame.Rect(x, y, 24, 32)
34         self.vx = 0; self.vy = 0; self.yerde = False
35         self.coyote_sayac = 0
36         self.buffer_sayac = 0
37
38     def zipla_istegi(self):
39         self.buffer_sayac = BUFFER_SURE
40
41     def guncelle(self, tuslar, katilar):
42         # Yatay
43         self.vx = 0
44         if tuslar[pygame.K_LEFT]: self.vx = -4
45         if tuslar[pygame.K_RIGHT]: self.vx = 4
46         self.rect.x += self.vx
47         for k in katilar:
48             if self.rect.colliderect(k):
49                 if self.vx > 0: self.rect.right = k.left
50                 elif self.vx < 0: self.rect.left = k.right
51
52         # Dikey + yercekimi
53         eski_yerde = self.yerde
54         self.vy = min(self.vy + YERCEKIMI, 10)
55         self.rect.y += int(self.vy)
```

```

56     self.yerde = False
57     for k in katilar:
58         if self.rect.colliderect(k):
59             if self.vy > 0:
60                 self.rect.bottom = k.top
61                 self.yerde = True
62             elif self.vy < 0:
63                 self.rect.top = k.bottom
64                 self.vy = 0
65
66     # Coyote
67     if eski_yerde and not self.yerde:
68         self.coyote_sayac = COYOTE_SURE
69     elif self.coyote_sayac > 0:
70         self.coyote_sayac -= 1
71
72     # Buffer
73     if self.buffer_sayac > 0:
74         self.buffer_sayac -= 1
75
76     if self.buffer_sayac > 0 and (self.yerde or self.coyote_sayac > 0):
77         self.vy = ZIP_HIZI
78         self.yerde = False
79         self.buffer_sayac = 0
80         self.coyote_sayac = 0
81
82     # ... ana dongu + cizim ...

```

Olay Akışı

KEYDOWN olayında `oyuncu.zipla_istegi()` çağır. Coyote sayacı yerde → havada geçişinde başlar; buffer sayacı KEYDOWN'da başlar; ikisi birlikte "zıplayabilir miyim?" kararını verir.

Talep 1: Görsel Feedback Ekle

Coyote aktifken oyuncu kutusunun etrafına altın sarı bir çerçeve çiz. Buffer aktifken aynı şekilde mavi bir çerçeve. İki durumun birlikte olabileceğini unutma (iki çerçeve birden görülsün).

Talep 2: HUD Sayaçları

Üst panelde her karede `f"coyote: {N} buffer: {M}"` yazısı göster. Test ederek mekaniklerin doğru zamanda başladığını gözle gör.

2 Çalışma 2: Variable Jump + Double Jump

Örnek Kod

`calisma2.py` dosyasına variable ve double jump özelliği ekleyin:

```

1  VAR_KES = 0.4
2  MAX_ZIPLAMA = 2
3  DOUBLE_ZIP_HIZI = -9
4
5  class Oyuncu:
6      def __init__(self, ...):
7          # ...
8          self.kalan_ziplama = MAX_ZIPLAMA
9          self.zipliyor = False
10
11     def tus_birakildi(self):

```

```

12     """KEYUP olayında cagrilir."""
13     if self.zipliyor and self.vy < 0:
14         self.vy *= VAR_KES
15         self.zipliyor = False
16
17     def guncelle(self, tuslar, katilar):
18         # ... fizik (Calisma 1'deki gibi) ...
19
20         # Yere deyince ziplamayi yenile
21         if self.yerde:
22             self.kalan_ziplama = MAX_ZIPLAMA
23
24         # Ziplama karari: buffer + (yerde/coyote/kalan hak)
25         if self.buffer_sayac > 0:
26             if self.yerde or self.coyote_sayac > 0:
27                 self.vy = ZIP_HIZI
28                 self.kalan_ziplama = MAX_ZIPLAMA - 1
29                 self.yerde = False
30                 self.zipliyor = True
31                 self.coyote_sayac = 0
32                 self.buffer_sayac = 0
33             elif self.kalan_ziplama > 0:
34                 self.vy = DOUBLE_ZIP_HIZI
35                 self.kalan_ziplama -= 1
36                 self.zipliyor = True
37                 self.buffer_sayac = 0
38
39         # Ana dongu olay yakala:
40         elif olay.type == pygame.KEYUP:
41             if olay.key in (pygame.K_SPACE, pygame.K_UP):
42                 oyuncu.tus_birakildi()

```

Talep 3: Triple Jump

`MAX_ZIPLAMA` değerini 3 yap. Artık oyuncu havada iki kere daha zıplayabilmeli. Yüksek tahtaların üstüne çıkabildiğini test et.

Talep 4: Variable Jump Çarpanı

`VAR_KES` değerini 0.0, 0.4, 0.7 ve 1.0 dene. Her değerde kısa-tıklama ve uzun-tutma arasındaki farkı yaz.

3 Çalışma 3: Düşman ve Stomping

Örnek Kod

`calisma3.py` dosyasına basit bir Patrol düşmanı ve üstten ezme:

```

1 class Dusman(pygame.sprite.Sprite):
2     def __init__(self, x, y, sol_x, sag_x, hiz=1.5):
3         super().__init__()
4         self.image = pygame.Surface((28, 28))
5         self.image.fill((220, 100, 60))
6         self.rect = self.image.get_rect(topleft=(x, y))
7         self.sol_x = sol_x; self.sag_x = sag_x
8         self.hiz = hiz
9         self.vx = hiz
10        self.vy = 0
11        self.oldurulebilir = True
12
13    def _ucurum_var(self, katilar):
14        if self.vx == 0: return False
15        test_x = self.rect.right+2 if self.vx > 0 else self.rect.left-6
16        test = pygame.Rect(test_x, self.rect.bottom+2, 4, 6)
17        for k in katilar:

```

```

18         if test.collidect(k): return False
19         return True
20
21     def guncelle(self, katilar):
22         if self._ucurum_var(katilar):
23             self.vx = -self.vx
24             elif self.rect.left <= self.sol_x and self.vx < 0:
25                 self.vx = self.hiz
26             elif self.rect.right >= self.sag_x and self.vx > 0:
27                 self.vx = -self.hiz
28         # fizik:
29         self.rect.x += int(self.vx)
30         for k in katilar:
31             if self.rect.collidect(k):
32                 if self.vx > 0: self.rect.right = k.left
33                 elif self.vx < 0: self.rect.left = k.right
34                 self.vx *= -1
35         self.vy = min(self.vy + 0.5, 10)
36         self.rect.y += int(self.vy)
37         for k in katilar:
38             if self.rect.collidect(k):
39                 if self.vy > 0: self.rect.bottom = k.top
40                 elif self.vy < 0: self.rect.top = k.bottom
41                 self.vy = 0
42
43     # Stomping:
44     def oyuncu_dusman(oyuncu, dusman_grubu):
45         for d in list(dusman_grubu):
46             if oyuncu.rect.collidect(d.rect):
47                 if (oyuncu.vy > 0 and
48                     oyuncu.rect.bottom < d.rect.top + 16 and
49                     d.oldurulebilir):
50                     d.kill()
51                     oyuncu.vy = -9
52                 else:
53                     oyuncu.can -= 1
54                     # i-frame baslat...

```

Talep 5: Chase AI

`ChaseDusman` sınıfı yaz. Oyuncu 200 piksel içine girince `durum = "chase"` olsun ve 3.0 hızla oyuncunun yönüne koşsun. Uzaklaşınca tekrar patrol'a dönsün.

Talep 6: I-Frames

Oyuncuda `iframe_sayac` sayacı ekle. Hasar alındığında 60'a set et. Her karede azalt; > 0 iken `hasar_al` çağrısını yok say. Sprite'ı on/off flicker ile çiz (her 4 karede bir gizle).

4 Bonus: Power-Up Ekle

Bonus Talep

`PowerUp(Sprite)` sınıfı yaz. Haritada üç renkli küp olsun:

- Mavi (hız): 10 saniye boyunca `MAX_HIZ` iki katı
- Sarı (zıplama): 10 saniye boyunca `MAX_ZIPLAMA = 3`
- Mor (kalkan): Bir sonraki hasarı iptal eder (15 saniye limit)

HUD'da aktif power-up'ları süre çubuğu olarak göster. Süre bittiğinde efekt silinsin.